Double Strike: Breaking Approximation-Based Side-Channel Countermeasures for DNNs

IEEE HOST 2026

Lorenzo CASALINO, Maria Mendez RÉAL, Jean-Christophe PRÉVOTET, Rubén SALVADOR

CentraleSupélec, IRISA, Inria (SUSHI Team)
name.surname@irisa.fr
Lab-STICC, Univ. Bretagne-Sud, UMR 6285
maria.mendez-real@univ-ubs.fr
Univ. Rennes, INSA Rennes, CNRS, IETR-UMR 6164
jean-christophe.prevotet@insa-rennes.fr

6 May 2026

ANR-21-CE39-0018

Table of Contents

1. Background
2. MACPruning
3. Methodology
4. Results
5. Conclusions

Deep Neural Networks (DNNs)

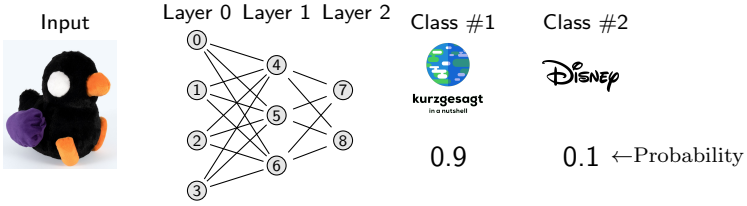
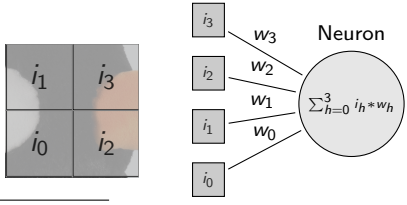
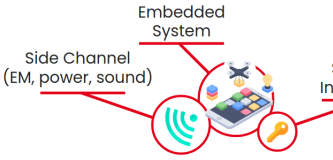


Figure: A DNN brand classifier[1].



[1]Duck and Kurzgesagt Logo belong to Kurzgesagt

Side-Channel Analysis



Embedded systems have *observable* effects (side channels) on the surrounding environment

The processed information Influences the side channel

Step #1 Observe Side Channel

Step #2 Statistically Analyse Side Channel

Step #3 Recover Information

An attacker can exploit side channels to recover information

Side-channel Driven Weight Extraction

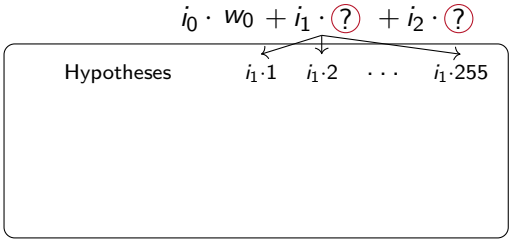
$$i_0 \cdot \textcircled{?} + i_1 \cdot \textcircled{?} + i_2 \cdot \textcircled{?}$$

the targets

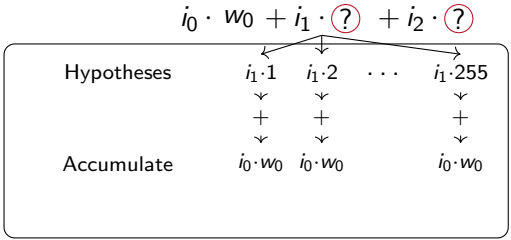
Side-channel Driven Weight Extraction

$$i_0 \cdot w_0 + i_1 \cdot \textcircled{?} + i_2 \cdot \textcircled{?}$$

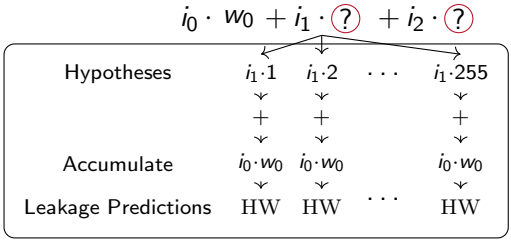
Side-channel Driven Weight Extraction



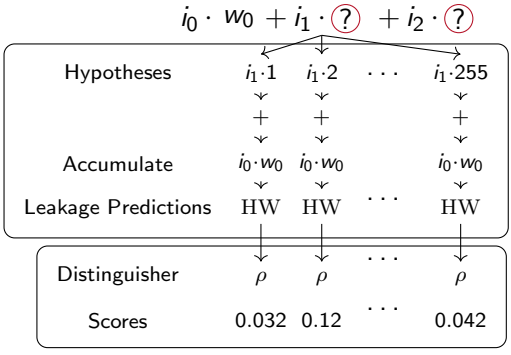
Side-channel Driven Weight Extraction



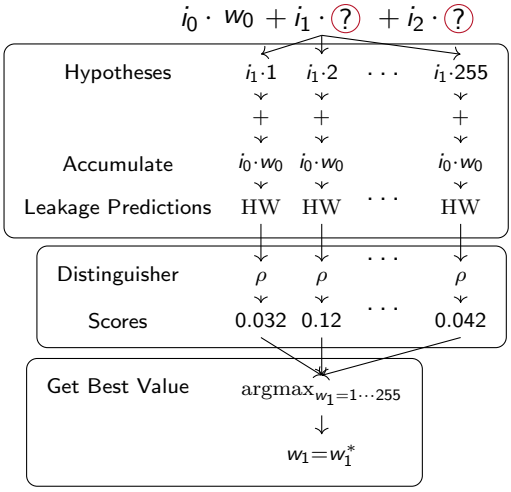
Side-channel Driven Weight Extraction



Side-channel Driven Weight Extraction



Side-channel Driven Weight Extraction



Countermeasures

Current proposal comes from the cryptoanalysis field:

Masking [Ath+22; Dub+22; Bro+24]

Replace the weight-dependent signal with N random ones

Shuffling [BPS22; Bre+23; Gan+23; Puš+25]

Randomly shuffle operations to bury weight-dependent signal in signal noise

DNN-Tailored Countermeasures

- But DNNs *are not* cryptosystems!
- **DNNs exhibit particular characteristics (e.g., error resilience)**

Table of Contents

1 Background

2 MACPruning

3 Methodology

4 Results

5 Conclusions

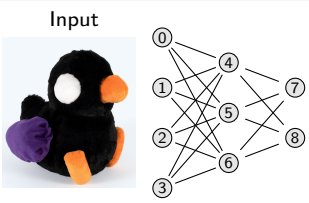
MACPruning Countermeasure [Din+25]

Approximate Computing [Leo+25]

Intentionally **trade accuracy for** *valuable* **resource gains**.

Dynamic Non-Important Input Pruning

At each inference, randomly skip (*prune*) **non-important** pixels.



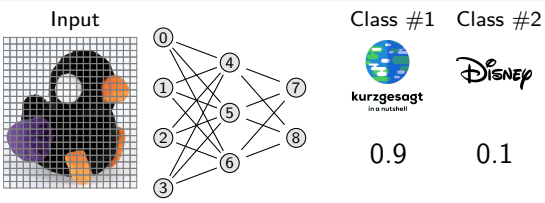
MACPruning Countermeasure [Din+25]

Approximate Computing [Leo+25]

Intentionally **trade accuracy for** *valuable* **resource gains**.

Dynamic Non-Important Input Pruning

At each inference, randomly skip (*prune*) **non-important** pixels.



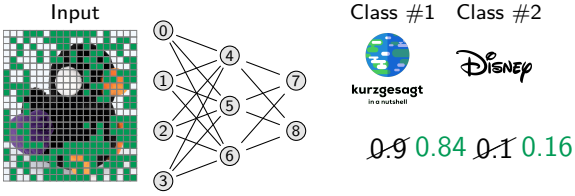
MACPruning Countermeasure [Din+25]

Approximate Computing [Leo+25]

Intentionally **trade accuracy for** *valuable* **resource gains**.

Dynamic Non-Important Input Pruning

At each inference, randomly skip (*prune*) **non-important** pixels.



- Non-important pixel: negligible accuracy reduction

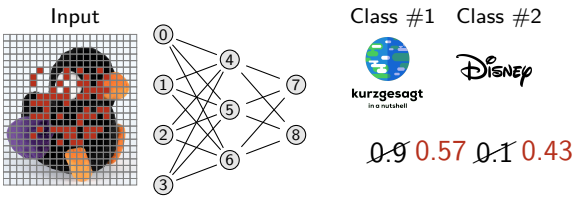
MACPruning Countermeasure [Din+25]

Approximate Computing [Leo+25]

Intentionally **trade accuracy for** *valuable* **resource gains**.

Dynamic Non-Important Input Pruning

At each inference, randomly skip (*prune*) **non-important** pixels.



- Non-important pixel: negligible accuracy reduction
- Important pixel: significant accuracy reduction

Explaining MACPruning' Security

- **The Idea**
  - Randomly skip (*prune*) non-important pixels.

Explaining MACPruning' Security

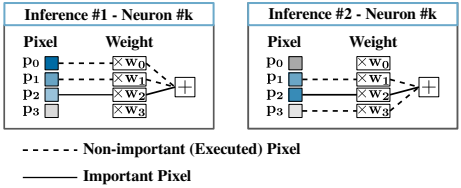
- **The Idea**
  - Randomly skip (*prune*) non-important pixels.
- **Security Implications**
  - Successful attack $\Longrightarrow$ exponential number of traces.

Explaining MACPruning' Security

- **The Idea**
  - Randomly skip (*prune*) non-important pixels.
- **Security Implications**
  - Successful attack $\Longrightarrow$ exponential number of traces.
- **Effects**

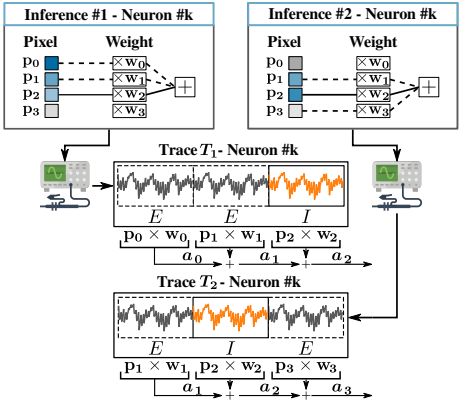
Explaining MACPruning' Security

- **The Idea**
  - Randomly skip (*prune*) non-important pixels.
- **Security Implications**
  - Successful attack $\Longrightarrow$ exponential number of traces.
- **Effects**
  - **Incorrect hypotheses predictions**,



Explaining MACPruning' Security

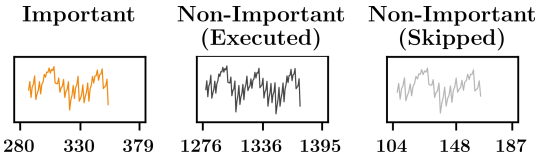
- **The Idea**
  - Randomly skip (*prune*) non-important pixels.
- **Security Implications**
  - Successful attack $\Longrightarrow$ exponential number of traces.
- **Effects**
  - **Incorrect hypotheses predictions**,
  - **Desynchronised traces**.



Control-flow Dependency and Side-channel Information

Input : *inputs*: $(w \times h)$-array of pixels
weights: $(w \times h)$-array of weights
Output: *acc*: accumulator

```
1 acc ← 0;
2 for i from 0 to length(inputs) do
3     if isImportant(i) then
4         acc ← inputs[i], weights[i] + acc;
5     else if isToExecute(i) then
6         acc ← inputs[i], weights[i] + acc;
7     else
          /* Skip pixel/MAC                    */
8 end
9 return acc
```



Side-Channel Information

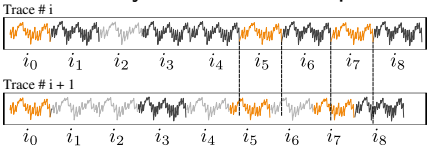
Control-flow dependency hints each pixel' importance

Question

What are the security implications?

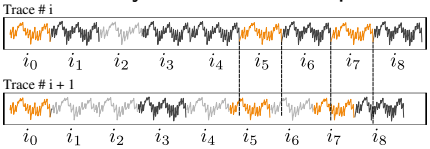
Control-flow Dependency and Security Implications

For each trace, we classify each trace sample and circumvent ...

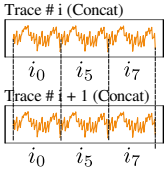


Control-flow Dependency and Security Implications

For each trace, we classify each trace sample and circumvent ...

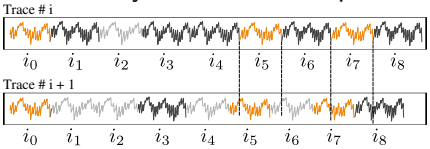


... *trace desynchronisation*.

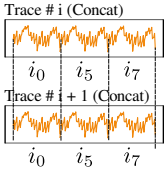


Control-flow Dependency and Security Implications

For each trace, we classify each trace sample and circumvent ...



... *trace desynchronisation*.



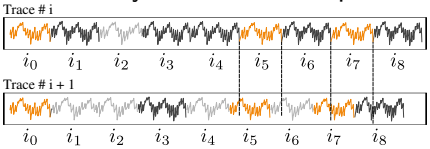
... *incorrect hypotheses prediction*.

$$\text{Trace}_i : i_0 \cdot w_0 + i_1 \cdot w_1 + \sum_{k=3}^{8} i_k \cdot w_k$$

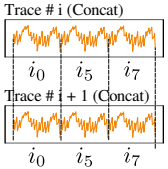
$$\text{Trace}_{i+1} : i_0 \cdot w_0 + i_3 \cdot w_3 + i_5 \cdot w_5 + i_7 \cdot w_7 + i_8 \cdot w_8$$

Control-flow Dependency and Security Implications

For each trace, we classify each trace sample and circumvent ...



... *trace desynchronisation*.



... *incorrect hypotheses prediction*.

$$\text{Trace}_i : i_0 \cdot w_0 + i_1 \cdot w_1 + \sum_{k=3}^{8} i_k \cdot w_k$$

$$\text{Trace}_{i+1} : i_0 \cdot w_0 + i_3 \cdot w_3 + i_5 \cdot w_5 + i_7 \cdot w_7 + i_8 \cdot w_8$$

Research Question

Can we circumvent the MACPruning countermeasure?

Table of Contents

1 Background

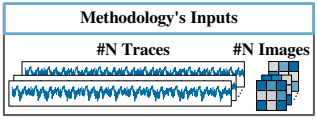
2 MACPruning

3 Methodology

4 Results

5 Conclusions

A Preprocessing Methodology to Circumvent MACPruning



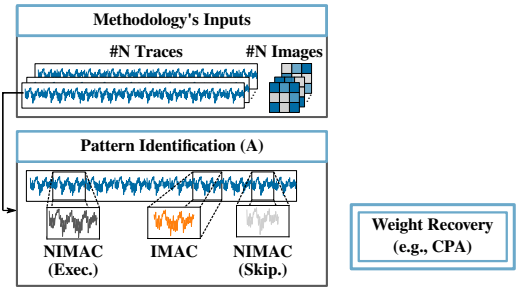
Goal

Recover the **important weights**.

Methodology

Figure: Preprocessing Methodology.

A Preprocessing Methodology to Circumvent MACPruning



Goal

Recover the **important weights**.

Methodology

A. Identify side-channel patterns;

Figure: Preprocessing Methodology.

A Preprocessing Methodology to Circumvent MACPruning

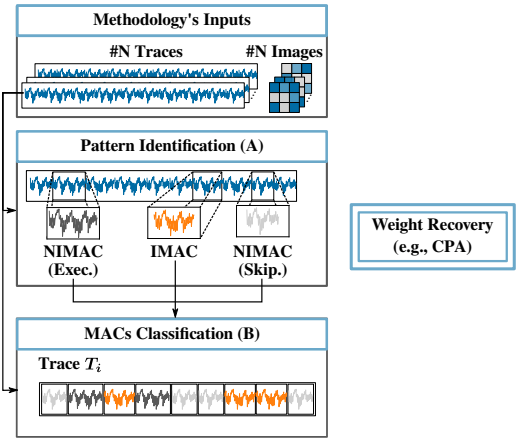


Figure: Preprocessing Methodology.

Goal

Recover the **important weights**.

Methodology

A. Identify side-channel patterns;

B. Identify inputs/MAC importance in traces;

A Preprocessing Methodology to Circumvent MACPruning

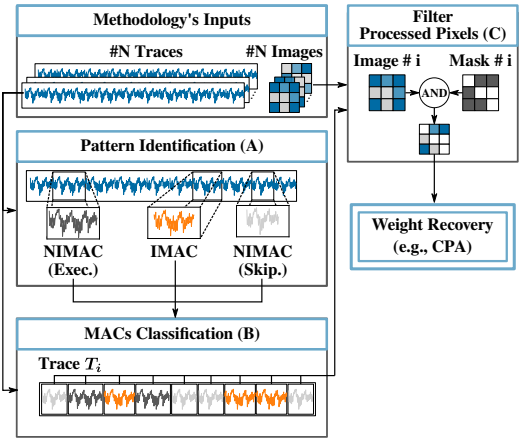


Figure: Preprocessing Methodology.

Goal

Recover the **important weights**.

Methodology

A. Identify side-channel patterns;
B. Identify inputs/MAC importance in traces;
C. Remove skipped pixels from images;

A Preprocessing Methodology to Circumvent MACPruning

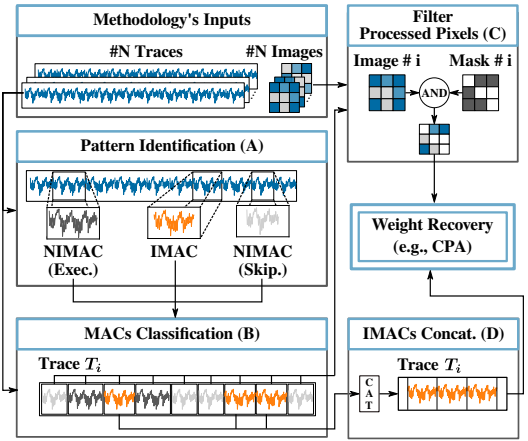


Figure: Preprocessing Methodology.

Goal

Recover the **important weights**.

Methodology

A. Identify side-channel patterns;
B. Identify inputs/MAC importance in traces;
C. Remove skipped pixels from images;
D. Preserve only the important MACs.

Table of Contents

1 Background

2 MACPruning

3 Methodology

4 Results

5 Conclusions

Experimental Setup

Deep Neural Network model

- Multi-Layer Preceptron
- 5 neurons in the first layer
- 32 pixels per neuron

Side-channel Setup

- ARM Cortex-M4 (Clk = 7.37MHz)
- Chipwhisperer Lite (Sampling rate = x4 Clk)

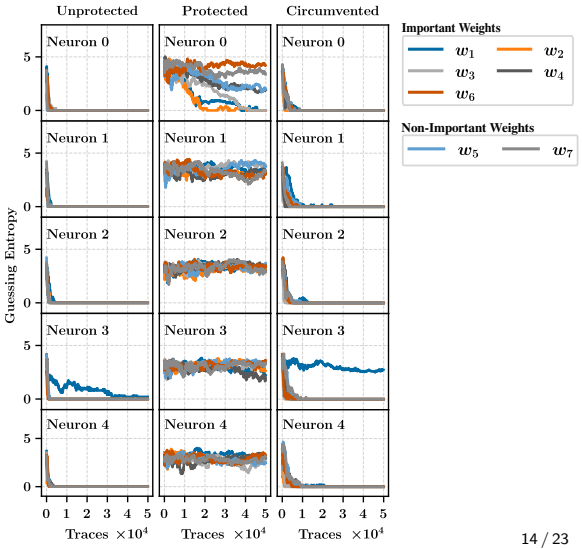
Experimental Procedure and Results

Experimental Procedure

1. Randomly and fix Important Pixels/MACs
2. Generate 5 sets of 50k random images
3. Recover 5 sets of 50k side-channel traces
4. Compute the *Guessing Entropy (GE)* of w_i:
   - Average position of the true weight value
   - If $GE(w_i) = 0 \rightarrow$ true weight scores the best

Results

- Recovered all the important weights



Summary Pt. 1

MACPruning:
Exponentially increases number of traces for successful weight extraction

Summary Pt. 1

MACPruning:
Exponentially increases number of traces for successful weight extraction

MACPruning Implementation:
Control-flow dependency (CFD) on pixel's importance

Summary Pt. 1

MACPruning:
Exponentially increases number of traces for successful weight extraction

MACPruning Implementation:
Control-flow dependency (CFD) on pixel's importance

Our Work:
Preprocessing methodology to exploit the CFD-induced side-channel leakage

Summary Pt. 1

MACPruning:
Exponentially increases number of traces for successful weight extraction

MACPruning Implementation:
Control-flow dependency (CFD) on pixel's importance

Our Work:
Preprocessing methodology to exploit the CFD-induced side-channel leakage

Results:
Extracted important weights with comparable number of traces of an unprotected MLP

Summary Pt. 1

Question:
What if we remove the control-flow dependency?

MACPruning: Fundamentally Broken?

Algorithm 4: Pseudo-code of a generic inference procedure enhanced with a control-flow-free MACPRUNING implementation.

Input : $inputs$: $(w \times h)$-array of pixels
$weights$: $(w \times h)$-array of weights
$IaPAM$: $(w \times h)$-array of bits
$randWord$: $(w \times h)$-array of bits
Output: acc: accumulator

```
1 acc ← 0;
2 for i from 0 to length(inputs) do
3 |   cond ← toSkip(IaPAM[i], randWord, i);
4 |   fn ← cond*addr(exec())+(1−cond)*addr(skip());
5 |   acc ← fn(inputs[i], weights[i], acc);
6 end
7 return acc;
```

A Branchless MACPruning

MACPruning: Fundamentally Broken?

Algorithm 4: Pseudo-code of a generic inference procedure enhanced with a control-flow-free MACPRUNING implementation.

Input : $inputs$: $(w \times h)$-array of pixels
$weights$: $(w \times h)$-array of weights
$IaPAM$: $(w \times h)$-array of bits
$randWord$: $(w \times h)$-array of bits
Output: acc: accumulator

```
1 acc ← 0;
2 for i from 0 to length(inputs) do
3 |   cond ← toSkip(IaPAM[i], randWord, i);
4 |   fn ← cond*addr(exec())+(1−cond)*addr(skip());
5 |   acc ← fn(inputs[i], weights[i], acc);
6 end
7 return acc;
```

A Branchless MACPruning

- `toSkip`: check whether to process pixel *i*
  - Store result in *cond*
  - Side-channel secure by hypothesis

MACPruning: Fundamentally Broken?

Algorithm 4: Pseudo-code of a generic inference procedure enhanced with a control-flow-free MACPRUNING implementation.

Input : $inputs$: $(w \times h)$-array of pixels
$weights$: $(w \times h)$-array of weights
$IaPAM$: $(w \times h)$-array of bits
$randWord$: $(w \times h)$-array of bits
Output: acc: accumulator

1 $acc \leftarrow 0$;
2 **for** i **from** 0 **to** `length`$(inputs)$ **do**
3 $cond \leftarrow$ `toSkip`$(IaPAM[i], randWord, i)$;
4 $fn \leftarrow cond{*}addr($`exec`$())+(1-cond){*}addr($`skip`$())$;
5 $acc \leftarrow fn(inputs[i], weights[i], acc)$;
6 **end**
7 **return** acc;

A Branchless MACPruning

- `toSkip`: check whether to process pixel *i*
  - Store result in *cond*
  - Side-channel secure by hypothesis
- `exec()`: execute the MAC and return updated `acc`
  - Process both important and non-important pixels
- `skip()`: simply return `acc`

MACPruning: Fundamentally Broken?

Algorithm 4: Pseudo-code of a generic inference procedure enhanced with a control-flow-free MACPRUNING implementation.

Input : $inputs$: $(w \times h)$-array of pixels
$weights$: $(w \times h)$-array of weights
$IaPAM$: $(w \times h)$-array of bits
$randWord$: $(w \times h)$-array of bits
Output: acc: accumulator

1 $acc \leftarrow 0$;
2 **for** i **from** 0 **to** `length`$(inputs)$ **do**
3 $cond \leftarrow$ `toSkip`$(IaPAM[i], randWord, i)$;
4 $fn \leftarrow cond{*}addr($`exec`$()){+}(1{-}cond){*}addr($`skip`$())$;
5 $acc \leftarrow fn(inputs[i], weights[i], acc)$;
6 **end**
7 **return** acc;

A Branchless MACPruning

- `toSkip`: check whether to process pixel *i*
  - Store result in *cond*
  - Side-channel secure by hypothesis
- `exec()`: execute the MAC and return updated `acc`
  - Process both important and non-important pixels
- `skip()`: simply return `acc`
- `exec()` **and** `skip` **different by design**

MACPruning: Fundamentally Broken?

Another Side-Channel Leakage

- `exec()` and `skip` different by design

MACPruning: Fundamentally Broken?

Another Side-Channel Leakage

- `exec()` and `skip` different by design
  - Different side-channel behaviour

MACPruning: Fundamentally Broken?

Another Side-Channel Leakage

- `exec()` and `skip` different by design
  - Different side-channel behaviour
- We know when we process a pixel

MACPruning: Fundamentally Broken?

Another Side-Channel Leakage

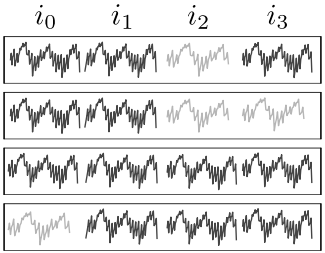
- `exec()` and `skip` different by design
  - Different side-channel behaviour
- We know when we process a pixel
  - `exec()`: process (non-)important pixels

MACPruning: Fundamentally Broken?

Another Side-Channel Leakage

- `exec()` and `skip` different by design
  - Different side-channel behaviour
- We know when we process a pixel
  - `exec()`: process (non-)important pixels
  - `skip()`: skip (non-)important pixels

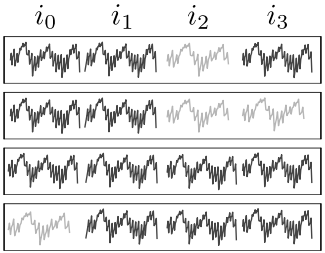
MACPruning: Fundamentally Broken?



Another Side-Channel Leakage

- `exec()` and `skip` different by design
  - Different side-channel behaviour
- We know when we process a pixel
  - `exec()`: process (non-)important pixels
  - `skip()`: skip (non-)important pixels

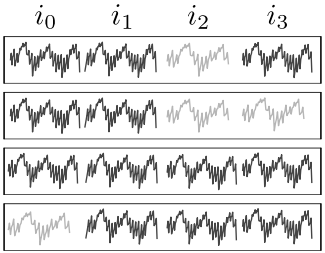
MACPruning: Fundamentally Broken?



Another Side-Channel Leakage

- `exec()` and `skip` different by design
  - Different side-channel behaviour
- We know when we process a pixel
  - `exec()`: process (non-)important pixels
  - `skip()`: skip (non-)important pixels
- **Goal**: recover important weights

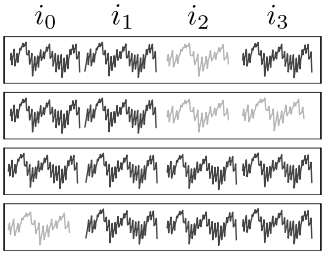
MACPruning: Fundamentally Broken?



Another Side-Channel Leakage

- `exec()` and `skip` different by design
  - Different side-channel behaviour
- We know when we process a pixel
  - `exec()`: process (non-)important pixels
  - `skip()`: skip (non-)important pixels
- **Goal**: recover important weights
- **Requirement**: identify important pixels

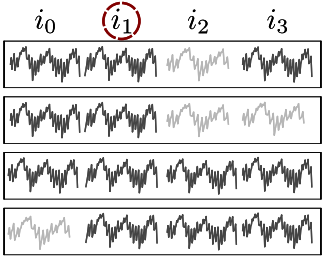
MACPruning: Fundamentally Broken?



Another Side-Channel Leakage

- `exec()` and `skip` different by design
  - Different side-channel behaviour
- We know when we process a pixel
  - `exec()`: process (non-)important pixels
  - `skip()`: skip (non-)important pixels
- **Goal**: recover important weights
- **Requirement**: identify important pixels
- We *infer* important pixels
  - Important pixel: **always** processed

MACPruning: Fundamentally Broken?



We may apply our preprocessing methodology

Another Side-Channel Leakage

- `exec()` and `skip` different by design
  - Different side-channel behaviour
- We know when we process a pixel
  - `exec()`: process (non-)important pixels
  - `skip()`: skip (non-)important pixels
- **Goal**: recover important weights
- **Requirement**: identify important pixels
- We *infer* important pixels
  - Important pixel: **always** processed

Table of Contents

1. Background
2. MACPruning
3. Methodology
4. Results
5. Conclusions

Summary Pt. 2

Branchless MACPruning:
Remove CFD-induced side-channel leakage

Summary Pt. 2

Branchless MACPruning:
Remove CFD-induced side-channel leakage

Still, we are pruning:
Control-flow depends on the pixel's importance

Summary Pt. 2

Branchless MACPruning:
Remove CFD-induced side-channel leakage

Still, we are pruning:
Control-flow depends on the pixel's importance

Pruning itself is leaking:
We can still rely on our preprocessing methodology

Summary Pt. 2

Branchless MACPruning:
Remove CFD-induced side-channel leakage

Still, we are pruning:
Control-flow depends on the pixel's importance

Pruning itself is leaking:
We can still rely on our preprocessing methodology

Results:
Pruning cannot be used as it is

Summary Pt. 2

Branchless MACPruning:
Remove CFD-induced side-channel leakage

Still, we are pruning:
Control-flow depends on the pixel's importance

Pruning itself is leaking:
We can still rely on our preprocessing methodology

Results:
Pruning cannot be used as it is

Is Everything Lost?

Can we use pruning-based countermeasures?

Is Everything Lost?

Can we use pruning-based countermeasures?

According to our observations ...

Is Everything Lost?

Can we use pruning-based countermeasures?

According to our observations ... **No**

Is Everything Lost?

Can we use pruning-based countermeasures?

According to our observations ... **No**

Unless we solve the pruning-induced leakage

Is Everything Lost?

Can we use pruning-based countermeasures?

According to our observations ... **No**
Unless we solve the pruning-induced leakage

Can Approximate Computing underpin side-channel countermeasures?

Is Everything Lost?

Can we use pruning-based countermeasures?

According to our observations ... **No**
Unless we solve the pruning-induced leakage

Can Approximate Computing underpin side-channel countermeasures?

Maybe

Is Everything Lost?

Can we use pruning-based countermeasures?

According to our observations ... **No**
Unless we solve the pruning-induced leakage

Can Approximate Computing underpin side-channel countermeasures?

Maybe

Approximate Computing $\Longrightarrow$ trade accuracy for performance
Secure Approximate Computing $\Longrightarrow$ trade accuracy for *security*

Thank You!

Questions?



Bibliography I

[Ath+22] Konstantinos Athanasiou et al. "Masking Feedforward Neural Networks Against Power Analysis Attacks". In: *Proceedings on Privacy Enhancing Technologies* (2022). ISSN: 2299-0984. URL: https://petsymposium.org/popets/2022/popets-2022-0025.php (visited on 03/12/2025).

[BPS22] Manuel Brosch, Matthias Probst, and Georg Sigl. "Counteract Side-Channel Analysis of Neural Networks by Shuffling". In: *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE). Mar. 2022, pp. 1305–1310. DOI: 10.23919/DATE54114.2022.9774710. URL: https://ieeexplore.ieee.org/abstract/document/9774710 (visited on 03/12/2025).

Bibliography II

[Bre+23] Jakub Breier et al. *A Desynchronization-Based Countermeasure Against Side-Channel Analysis of Neural Networks*. Mar. 25, 2023. DOI: 10.48550/arXiv.2303.18132. arXiv: 2303.18132 [cs]. URL: http://arxiv.org/abs/2303.18132 (visited on 03/12/2025). Pre-published.

[Bro+24] Manuel Brosch et al. "A Masked Hardware Accelerator for Feed-Forward Neural Networks With Fixed-Point Arithmetic". In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 32.2 (Feb. 2024), pp. 231–244. ISSN: 1557-9999. DOI: 10.1109/TVLSI.2023.3340553. URL: https://ieeexplore.ieee.org/abstract/document/10359503 (visited on 03/12/2025).

Bibliography III

[Din+25] Ruyi Ding et al. "MACPruning: Dynamic Operation Pruning to Mitigate Side-Channel DNN Model Extraction". In: *2025 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 2025 IEEE International Symposium on Hardware Oriented Security and Trust (HOST). May 2025, pp. 12–22. DOI: 10.1109/HOST64725.2025.11050072. URL: https://ieeexplore.ieee.org/document/11050072 (visited on 10/05/2025).

Bibliography IV

[Dub+22] Anuj Dubey et al. "ModuloNET: Neural Networks Meet Modular Arithmetic for Efficient Hardware Masking". In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2022), pp. 506–556. ISSN: 2569-2925. DOI: 10.46586/tches.v2022.i1.506-556. URL: https://tches.iacr.org/index.php/TCHES/article/view/9306 (visited on 03/12/2025).

[Gan+23] Karthik Ganesan et al. *BlackJack: Secure Machine Learning on IoT Devices through Hardware-Based Shuffling*. Oct. 26, 2023. DOI: 10.48550/arXiv.2310.17804. arXiv: 2310.17804 [cs]. URL: http://arxiv.org/abs/2310.17804 (visited on 10/05/2025). Pre-published.

Bibliography V

[Jap+25] Aditya Japa et al. "Security of Approximate Neural Networks against Power Side-channel Attacks". In: *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. 2025 62nd ACM/IEEE Design Automation Conference (DAC). June 2025, pp. 1–7. DOI: 10.1109/DAC63849.2025.11133333. URL: https://ieeexplore.ieee.org/document/11133333/ (visited on 09/29/2025).

[Leo+25] Vasileios Leon et al. "Approximate Computing Survey, Part I: Terminology and Software & Hardware Approximation Techniques". In: *ACM Comput. Surv.* 57.7 (Mar. 5, 2025), 185:1–185:36. ISSN: 0360-0300. DOI: 10.1145/3716845. URL: https://dl.acm.org/doi/10.1145/3716845 (visited on 04/22/2025).

Bibliography VI

[Puš+25] Leonard Puškáč et al. "Make Shuffling Great Again: A Side-Channel-Resistant Fisher–Yates Algorithm for Protecting Neural Networks". In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 33.7 (July 2025), pp. 1959–1971. ISSN: 1557-9999. DOI: 10.1109/TVLSI.2025.3564357. URL: https://ieeexplore.ieee.org/abstract/document/10988891 (visited on 07/25/2025).

Bibliography VII

[Zha+26] Lu Zhang et al. "ApproPower: Characterizing the Power Side-Channel Features of Approximate Multipliers with Symbolic Path Analysis". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2026), pp. 1–1. ISSN: 1937-4151. DOI: 10.1109/TCAD.2026.3656474. URL: https://ieeexplore.ieee.org/abstract/document/11359193 (visited on 01/26/2026).

Microarchitectural Leakage

Unsuccessful Attacks?

- Not possible to recover the weight w_7
- Why?
- Solution: consider extended traces

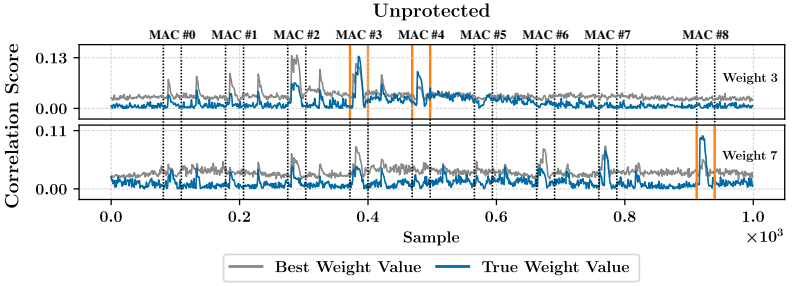


Figure: Microarchitectural Leakage.

Recovery of Non-Important Weights

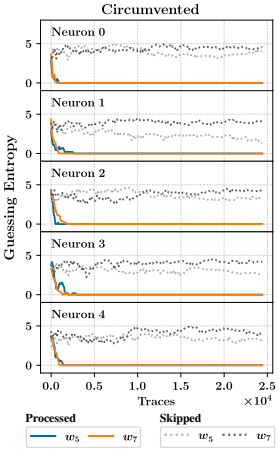


Figure: GE Filtered Traces.

Information carrying

- Partition traces in two sets:
  - *Processed*: non-important weight w_i processed
  - *Skipped*: non-important weight w_i skipped
- Compute GE on both sets
- Filtered traces carry information on non-important weights